

LAPORAN PRAKTIKUM

STRUKTUR DATA

“TREE DAN GRAPH”



Disusun oleh:

Ndaru Pradana Gatot Suseno

2411532007

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Jovantri Immanuel Gulo

**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena laporan Praktikum Struktur Data 9 dapat diselesaikan. Laporan ini mendokumentasikan pembelajaran mengenai struktur data tree dan graph serta algoritma penelusurannya dalam bahasa Java.

Praktikum membahas pembentukan binary tree, traversal preorder, inorder, dan postorder, representasi graph menggunakan adjacency list, serta penelusuran DFS dan BFS. Laporan tugas menerapkan konsep graph dalam aplikasi Peta Harian berbasis Java Swing.

Penulis menyampaikan terima kasih kepada dosen pengampu dan asisten praktikum atas bimbingan selama pelaksanaan praktikum. Isi laporan disesuaikan dengan source code pada package pekan9_2411532007 dan hasil pengujian program.

Penulis menyadari bahwa laporan ini masih dapat dikembangkan. Oleh sebab itu, kritik dan saran yang membangun sangat diharapkan.

Ndaru Pradana G.S

DAFTAR ISI

Contents

KATA PENGANTAR	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	4
Latar Belakang	4
Tujuan Praktikum.....	4
Manfaat Praktikum.....	4
BAB II PEMBAHASAN	5
Dasar Teori.....	5
Class Node_2411532007 dan Btree_2411532007	6
Class Btree_2411532007	8
Program BtreeDriver_2411532007	10
Program GraphTraversal_2411532007	12
BAB III KESIMPULAN.....	14
DAFTAR PUSTAKA	15
LAPORAN TUGAS	16
Tujuan Tugas.....	16
Class petaHarian_2411532007	17
Hasil Eksekusi Laporan Tugas.....	22
Analisis Hasil	22
Kesimpulan Laporan Tugas	22
Saran.....	23

BAB I

PENDAHULUAN

Latar Belakang

Tree dan graph merupakan struktur data nonlinier yang digunakan untuk merepresentasikan hubungan antarelemen. Tree membentuk hubungan hierarkis tanpa siklus, sedangkan graph mampu menggambarkan hubungan yang lebih umum melalui kumpulan vertex dan edge.

Praktikum kesembilan mempelajari binary tree beserta traversal preorder, inorder, dan postorder. Materi graph mencakup representasi adjacency list serta penelusuran Depth First Search dan Breadth First Search. Konsep tersebut kemudian diterapkan pada peta lokasi untuk mencari jalur antara titik awal dan tujuan.

Tujuan Praktikum

1. Memahami konsep node, root, child, leaf, dan subtree pada binary tree.
2. Menerapkan traversal preorder, inorder, dan postorder.
3. Merepresentasikan graph menggunakan adjacency list.
4. Menerapkan DFS dan BFS untuk menelusuri graph.
5. Mengembangkan aplikasi peta berbasis graph dengan visualisasi jalur.

Manfaat Praktikum

Praktikum membantu memahami representasi hubungan hierarkis dan jaringan, penggunaan rekursi, stack, serta queue pada proses traversal. Konsep tersebut dapat diterapkan pada sistem navigasi, jaringan komputer, struktur direktori, dan pencarian jalur.

BAB II

PEMBAHASAN

Dasar Teori

Tree adalah struktur data nonlinier yang terdiri atas node dan edge. Node paling atas disebut root, sedangkan node tanpa child disebut leaf. Binary tree membatasi setiap node memiliki paling banyak dua child, yaitu left dan right.

Traversal tree merupakan proses mengunjungi seluruh node. Preorder mengunjungi root-kiri-kanan, inorder mengunjungi kiri-root-kanan, dan postorder mengunjungi kiri-kanan-root.

Graph terdiri atas vertex dan edge. Graph tidak berarah memiliki hubungan dua arah sehingga setiap edge dapat dilalui dari kedua vertex yang terhubung. Adjacency list menyimpan daftar tetangga untuk setiap vertex.

Depth First Search menelusuri graph sedalam mungkin sebelum kembali ke cabang sebelumnya dan lazim menggunakan stack atau rekursi. Breadth First Search mengunjungi vertex per tingkat dengan bantuan queue dan dapat menemukan jalur dengan jumlah edge minimum pada graph tidak berbobot.

Dalam visualisasi peta, vertex merepresentasikan lokasi, sedangkan edge merepresentasikan hubungan atau rute langsung antarlokasi.

Class Node_2411532007 dan Btree_2411532007

Node_2411532007 menyimpan data serta referensi child kiri dan kanan. Btree_2411532007 mengelola root, penambahan node, perhitungan jumlah node, dan pemanggilan traversal.

Source Code

```

1 package pekan9_2411532007;
2
3 public class Node_2411532007 {
4     int data_2007;
5     Node_2411532007 left_2007;
6     Node_2411532007 right_2007;
7
8     public Node_2411532007(int data_2007) {
9         this.data_2007 = data_2007;
10        left_2007 = null;
11        right_2007 = null;
12    }
13
14    public void setLeft_2007(Node_2411532007 node_2007) {
15        if ( left_2007 == null ) {
16            left_2007 = node_2007;
17        }
18    }
19
20    public void setRight_2007(Node_2411532007 node_2007) {
21        if (right_2007 == null ) {
22            right_2007 = node_2007;
23        }
24    }
25
26    public Node_2411532007 getleft_2007() {
27        return left_2007;
28    }
29
30    public Node_2411532007 getRight_2007() {
31        return right_2007;
32    }
33
34    public int getData_2007() {
35        return data_2007;
36    }
37
38    public void setData_2007(int data_2007) {
39        this.data_2007 = data_2007;
40    }
41
42    void printPreorder_2007(Node_2411532007 node_2007) {
43        if(node_2007 == null ) {
44            if(node_2007 == null ) {
45                return;
46            }
47            System.out.print(node_2007.data_2007 + " ");
48            printPreorder_2007(node_2007.left_2007);
49            printPreorder_2007(node_2007.right_2007);
50        }
51
52    void printPostorder_2007(Node_2411532007 node_2007) {
53        if(node_2007 == null ) {
54            return;
55        }
56        printPostorder_2007(node_2007.left_2007);
57        printPostorder_2007(node_2007.right_2007);
58        System.out.print(node_2007.data_2007 + " ");
59    }
60
61    void printInorder_2007(Node_2411532007 node_2007) {
62        if(node_2007 == null ) {
63            return;
64        }
65        printInorder_2007(node_2007.left_2007);
66        System.out.print(node_2007.data_2007 + " ");
67        printInorder_2007(node_2007.right_2007);
68    }
69
70    public String print_2007() {
71        return this.print_2007("", true, "");
72    }
73
74    public String print_2007(String prefix_2007, boolean isTail_2007, String sb_2007) {
75        if (right_2007 != null ) {
76            right_2007.print_2007(prefix_2007 + (isTail_2007 ? "| " : " "), false, sb_2007);
77        }
78        System.out.println( prefix_2007 + (isTail_2007 ? "\\-- " : "/-- ") + data_2007);
79        if (left_2007 != null ) {
80            left_2007.print_2007(prefix_2007 + (isTail_2007 ? " " : "| "), true, sb_2007);
81        }
82        return sb_2007;
83    }
84 }
85

```

Penjelasan Operasi

1. Atribut `data_2007` menyimpan nilai node.
2. `left_2007` dan `right_2007` menyimpan referensi child.
3. Method pada Btree mengatur root dan menambahkan node sesuai posisi.
4. `countNodes_2007()` menghitung jumlah simpul secara rekursif.
5. Traversal diteruskan kepada method pada class Node.

Analisis

Pemisahan class Node dan Btree membuat representasi data dan operasi tree lebih modular. Setiap node dapat membentuk subtree melalui referensi kiri dan kanan.

Class Btree_2411532007

Class Btree_2411532007 menyediakan operasi dasar untuk binary tree dan menghubungkan object root dengan method traversal.

Source Code

```

1 package pekan9_2411532007;
2
3 public class Btree_2411532007 {
4     private Node_2411532007 root_2007;
5     private Node_2411532007 currentNode_2007;
6
7     public Btree_2411532007() {
8         root_2007 = null;
9     }
10
11     public boolean search_2007(int data_2007) {
12         return search_2007(root_2007, data_2007);
13     }
14
15     private boolean search_2007(Node_2411532007 node_2007, int data_2007) {
16         if (node_2007.getData_2007() == data_2007)
17             return true;
18         if (node_2007.getLeft_2007() != null)
19             if (search_2007(node_2007.getLeft_2007(), data_2007))
20                 return true;
21         if (node_2007.getRight_2007() != null)
22             if (search_2007(node_2007.getRight_2007(), data_2007))
23                 return true;
24         return false;
25     }
26
27     public void printInorder_2007() {
28         root_2007.printInorder_2007(root_2007);
29     }
30
31     public void printPreorder_2007() {
32         root_2007.printPreorder_2007(root_2007);
33     }
34     public void printPostorder_2007() {
35         root_2007.printPostorder_2007(root_2007);
36     }
37
38     public Node_2411532007 getRoot_2007() {
39         return root_2007;
40     }
41
42     public boolean isEmpty_2007() {
43         return root_2007 == null;
44     }
45
46     public int countNodes_2007() {
47         return countNodes_2007(root_2007);
48     }
49
50     private int countNodes_2007(Node_2411532007 node_2007) {
51         int count_2007 = 1;
52         if (node_2007 == null) {
53             return 0;
54         } else {
55             count_2007 += countNodes_2007(node_2007.getLeft_2007());
56             count_2007 += countNodes_2007(node_2007.getRight_2007());
57             return count_2007;
58         }
59     }
60
61     public void print_2007() {
62         root_2007.print_2007();
63     }
64
65     public Node_2411532007 getCurrent_2007() {
66         return currentNode_2007;
67     }
68
69     public void setCurrent_2007(Node_2411532007 node_2007) {
70         this.currentNode_2007 = node_2007;
71     }
72
73     public void setRoot_2007(Node_2411532007 root_2007) {
74         this.root_2007 = root_2007;
75     }
76 }

```

Penjelasan Operasi

1. isEmpty_2007() memeriksa apakah root belum tersedia.
2. insert_2007() menambahkan data berdasarkan posisi yang diberikan.

3. `countNodes_2007()` menghitung seluruh node.
4. `printInorder_2007()`, `printPreorder_2007()`, dan `printPostorder_2007()` memulai traversal.
5. `print_2007()` menampilkan bentuk pohon melalui representasi teks.

Analisis

Class `Btree` bertindak sebagai pengendali struktur tree. Operasi traversal menggunakan `root` sebagai titik awal dan memanfaatkan pemanggilan rekursif pada setiap subtree.

Program BtreeDriver_2411532007

BtreeDriver_2411532007 membangun tree berisi sembilan node, menghitung simpul, menjalankan tiga traversal, dan menampilkan bentuk tree.

Source Code

```

1 package pekan9_2411532007;
2
3 public class BtreeDriver_2411532007 {
4
5     public static void main(String[] args) {
6
7         Btree_2411532007 tree_2007 = new Btree_2411532007();
8         System.out.print("Jumlah Simpul awal pohon: ");
9         System.out.println(tree_2007.countNodes_2007());
10
11         Node_2411532007 root_2007 = new Node_2411532007(1);
12
13         tree_2007.setRoot_2007(root_2007);
14         System.out.println("Jumlah simpul jika hanya ada root");
15         System.out.println(tree_2007.countNodes_2007());
16         Node_2411532007 node2_2007 = new Node_2411532007(2);
17         Node_2411532007 node3_2007 = new Node_2411532007(3);
18         Node_2411532007 node4_2007 = new Node_2411532007(4);
19         Node_2411532007 node5_2007 = new Node_2411532007(5);
20         Node_2411532007 node6_2007 = new Node_2411532007(6);
21         Node_2411532007 node7_2007 = new Node_2411532007(7);
22         Node_2411532007 node8_2007 = new Node_2411532007(8);
23         Node_2411532007 node9_2007 = new Node_2411532007(9);
24         root_2007.setLeft_2007(node2_2007);
25         node2_2007.setLeft_2007(node4_2007);
26         node2_2007.setRight_2007(node5_2007);
27         node4_2007.setRight_2007(node8_2007);
28         root_2007.setRight_2007(node3_2007);
29         node3_2007.setLeft_2007(node6_2007);
30         node3_2007.setRight_2007(node7_2007);
31         node6_2007.setLeft_2007(node9_2007);
32
33         tree_2007.setCurrent_2007(tree_2007.getRoot_2007());
34         System.out.println("menampilkan simpul terakhir: ");
35         System.out.println(tree_2007.getCurrent_2007().getData_2007());
36         System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
37         System.out.println(tree_2007.countNodes_2007());
38         System.out.println("InOrder: ");
39         tree_2007.printInorder_2007();
40         System.out.println("\nPreorder: ");
41         tree_2007.printPreorder_2007();
42         System.out.println("\nPostorder: ");
43         tree_2007.printPostorder_2007();
44         System.out.println("\nMenampilkan simpul dalam bentuk pohon");
45         tree_2007.print_2007();
46     }
47 }
48
49

```

Penjelasan Operasi

1. Object tree dibuat dalam keadaan kosong.
2. Node dimasukkan pada posisi kiri dan kanan sehingga membentuk binary tree.
3. Jumlah simpul diperiksa sebelum dan setelah penambahan.
4. Traversal inorder, preorder, dan postorder dijalankan.
5. Struktur tree dicetak dalam bentuk cabang teks.

Hasil Eksekusi

```

terminated> BtreeDriver_2411532007 [Java Application] C:\Users\ndaru\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_23.0.1.v20241024-1700\jre\bin\javaw.exe (4 Jul 2026
umlah Simpul awal pohon: 0
umlah simpul jika hanya ada root

tampilkan simpul terakhir:

umlah simpul; setelah simpul 7 ditambahkan

InOrder:
8 2 5 1 9 6 3 7
Preorder:
2 4 8 5 3 6 9 7
Postorder:
4 5 2 9 6 7 3 1
tampilkan simpul dalam bentuk pohon
    /-- 7
    /-- 3
    | \-- 6
    |  \-- 9
-- 1
  | /-- 5
  | \-- 2
  | /-- 8
  | \-- 4

```

Analisis

Output menunjukkan sembilan node berhasil dibentuk. Urutan inorder, preorder, dan postorder berbeda karena posisi kunjungan root pada setiap traversal tidak sama. Bentuk cabang teks memudahkan pemeriksaan hubungan parent-child.

Program GraphTraversal_2411532007

GraphTraversal_2411532007 membentuk graph tidak berarah dengan adjacency list lalu menjalankan DFS dan BFS dari vertex A.

Source Code

```

1 package pekan9_2411532007;
2
3 import java.util.Map;
4
5
6
7
8
9
10
11
12
13 public class GraphTraversal_2411532007 {
14     private Map<String, List<String>> graph_2007 = new HashMap<>();
15
16
17     public void addEdge_2007(String node1_2007, String node2_2007) {
18         graph_2007.putIfAbsent(node1_2007, new ArrayList<>());
19         graph_2007.putIfAbsent(node2_2007, new ArrayList<>());
20         graph_2007.get(node1_2007).add(node2_2007);
21         graph_2007.get(node2_2007).add(node1_2007);
22     }
23
24     public void printGraph_2007() {
25         System.out.println("Graf Awal (Adjacency List):");
26         for (String node_2007 : graph_2007.keySet()) {
27             System.out.print(node_2007 + " -> ");
28             List<String> neighbors_2007 = graph_2007.get(node_2007);
29             System.out.println(String.join(", ", neighbors_2007));
30         }
31         System.out.println();
32     }
33
34     public void dfs_2007(String start_2007) {
35         Set<String> visited_2007 = new HashSet<>();
36         System.out.println("Penelusuran DFS:");
37         dfsHelper_2007(start_2007, visited_2007);
38         System.out.println();
39     }
40
41     private void dfsHelper_2007(String current_2007, Set<String> visited_2007) {
42         if (visited_2007.contains(current_2007))
43             return;
44         visited_2007.add(current_2007);
45         System.out.print(current_2007 + " ");
46         for (String neighbor_2007 : graph_2007.getOrDefault(current_2007, new ArrayList<>())) {
47             dfsHelper_2007(neighbor_2007, visited_2007);
48         }
49     }
50
51     public void bfs_2007(String start_2007) {
52         Set<String> visited_2007 = new HashSet<>();
53         Queue<String> queue_2007 = new LinkedList<>();
54         queue_2007.add(start_2007);
55         visited_2007.add(start_2007);
56         System.out.println("Penelusuran BFS:");
57         while (!queue_2007.isEmpty()) {
58             String current_2007 = queue_2007.poll();
59             System.out.print(current_2007 + " ");
60             for (String neighbor_2007 : graph_2007.getOrDefault(current_2007, new ArrayList<>())) {
61                 if (!visited_2007.contains(neighbor_2007)) {
62                     queue_2007.add(neighbor_2007);
63                     visited_2007.add(neighbor_2007);
64                 }
65             }
66         }
67         System.out.println();
68     }
69
70     public static void main(String[] args) {
71         GraphTraversal_2411532007 graph_2007 = new GraphTraversal_2411532007();
72
73         graph_2007.addEdge_2007("A", "B");
74         graph_2007.addEdge_2007("A", "C");
75         graph_2007.addEdge_2007("B", "D");
76         graph_2007.addEdge_2007("B", "E");
77
78         System.out.println("Graf Awal adalah: ");
79         graph_2007.printGraph_2007();
80
81         graph_2007.dfs_2007("A");
82         graph_2007.bfs_2007("A");
83     }
84 }
85
86
87

```

Penjelasan Operasi

1. addVertex_2007() menambahkan vertex beserta daftar tetangga kosong.
2. addEdge_2007() menambahkan hubungan dua arah.

3. `dfs_2007()` menggunakan rekursi dan himpunan `visited`.
4. `bfs_2007()` menggunakan Queue untuk penelusuran per tingkat.
5. `printGraph_2007()` menampilkan adjacency list.

Hasil Eksekusi

```
Graf Awal adalah:  
Graf Awal (Adjacency List):  
A -> B, C  
B -> A, D, E  
C -> A  
D -> B  
E -> B  
  
Penelusuran DFS:  
A B D E C  
Penelusuran BFS:  
A B C D E
```

Analisis

DFS menghasilkan urutan A, B, D, E, C karena penelusuran mengikuti satu cabang lebih dahulu. BFS menghasilkan A, B, C, D, E karena seluruh tetangga terdekat dikunjungi sebelum vertex pada tingkat berikutnya.

BAB III

KESIMPULAN

Praktikum kesembilan menunjukkan cara membangun binary tree dan graph dalam Java. Traversal tree memberikan urutan kunjungan berbeda sesuai posisi root, sedangkan DFS dan BFS menggunakan strategi penelusuran yang berbeda.

Penerapan pada aplikasi peta memperlihatkan bahwa graph dapat digunakan untuk merepresentasikan lokasi dan hubungan rute serta menampilkan jalur hasil pencarian secara visual.

DAFTAR PUSTAKA

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Java (6th ed.). Wiley.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Sierra, K., Bates, B., & Gee, T. (2022). Head First Java (3rd ed.). O'Reilly Media.

Oracle. (2024). Java Platform, Standard Edition API Specification. Oracle Corporation.

LAPORAN TUGAS

Laporan tugas mengembangkan petaHarian_2411532007, yaitu aplikasi peta lokasi kampus dan lingkungan sekitar yang menyediakan pencarian jalur menggunakan BFS dan DFS.

Tujuan Tugas

1. Merepresentasikan lokasi sebagai vertex dan hubungan jalan sebagai edge.
2. Menampilkan graph dalam panel grafis.
3. Mencari jalur antara lokasi awal dan tujuan menggunakan BFS dan DFS.
4. Menampilkan jalur, node yang dikunjungi, dan jumlah kunjungan.
5. Membedakan node jalur dan node yang hanya dikunjungi melalui warna.

Class petaHarian_2411532007

Class petaHarian_2411532007 merupakan JFrame yang membangun graph peta, menyediakan pilihan lokasi, tombol BFS/DFS/Reset, label hasil, dan panel visualisasi.

Source Code

```

1 package pekan9_2411532007;
2
3 import javax.swing.*;
4
5
6
7
8
9
10 public class petaHarian_2411532007 extends JFrame {
11
12     private JComboBox<String> comboBoxAwal_2007;
13     private JComboBox<String> comboBoxAkhir_2007;
14     private JButton btnBfs_2007;
15     private JButton btnDfs_2007;
16     private JButton btnReset_2007;
17     private JLabel lblJalur_2007;
18     private JLabel lblNodeDikunjungi_2007;
19     private JLabel lblJumlahNode_2007;
20     private GraphPanel_2007 panelGrafik_2007;
21
22     private Map<String, Vertex_2007> vertices_2007;
23     private Map<String, List<String>> adjacencyList_2007;
24     private List<String> visitedNodes_2007;
25     private List<String> pathNodes_2007;
26
27     public static void main(String[] args) {
28         EventQueue.invokeLater() -> {
29             try {
30                 petaHarian_2411532007 frame = new petaHarian_2411532007();
31                 frame.setVisible(true);
32             } catch (Exception e) {
33                 e.printStackTrace();
34             }
35         });
36     }
37
38     public petaHarian_2411532007() {
39         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
40         setSize(583, 700);
41         setLocationRelativeTo(null);
42         getContentPane().setLayout(null);
43
44         initializeGraph_2007();
45
46         JLabel lblLokasiAwal_2007 = new JLabel("Lokasi Awal:");
47         lblLokasiAwal_2007.setBounds(10, 11, 80, 14);
48         getContentPane().add(lblLokasiAwal_2007);
49
50         comboBoxAwal_2007 = new JComboBox<>();
51         comboBoxAwal_2007.setBounds(90, 7, 150, 22);
52         for (String vertex : vertices_2007.keySet()) {
53             comboBoxAwal_2007.addItem(vertex);
54         }
55         getContentPane().add(comboBoxAwal_2007);
56
57         JLabel lblLokasiAkhir_2007 = new JLabel("Lokasi Tujuan:");
58         lblLokasiAkhir_2007.setBounds(10, 36, 80, 14);
59         getContentPane().add(lblLokasiAkhir_2007);
60
61         comboBoxAkhir_2007 = new JComboBox<>();
62         comboBoxAkhir_2007.setBounds(90, 32, 150, 22);
63         for (String vertex : vertices_2007.keySet()) {
64             comboBoxAkhir_2007.addItem(vertex);
65         }
66         getContentPane().add(comboBoxAkhir_2007);
67
68         btnBfs_2007 = new JButton("BFS");
69         btnBfs_2007.setBounds(260, 7, 80, 23);
70         btnBfs_2007.addActionListener(e -> BFS_2007());
71         getContentPane().add(btnBfs_2007);
72
73         btnDfs_2007 = new JButton("DFS");
74         btnDfs_2007.setBounds(350, 7, 80, 23);
75         btnDfs_2007.addActionListener(e -> DFS_2007());
76         getContentPane().add(btnDfs_2007);
77
78         btnReset_2007 = new JButton("RESET");
79         btnReset_2007.setBounds(440, 7, 80, 23);
80         btnReset_2007.addActionListener(e -> resetGraph_2007());
81         getContentPane().add(btnReset_2007);
82
83         panelGrafik_2007 = new GraphPanel_2007();
84         panelGrafik_2007.setBounds(10, 61, 500, 500);
85         getContentPane().add(panelGrafik_2007);
86
87         JLabel lblHasilPencarian_2007 = new JLabel("Hasil Pencarian:");
88         lblHasilPencarian_2007.setBounds(10, 563, 150, 14);
89         getContentPane().add(lblHasilPencarian_2007);
90

```

```

90
91     lblJalur_2007 = new JLabel("Jalur: -");
92     lblJalur_2007.setBounds(10, 577, 780, 14);
93     getContentPane().add(lblJalur_2007);
94
95     lblNodeDikunjungi_2007 = new JLabel("Node Dikunjungi: -");
96     lblNodeDikunjungi_2007.setBounds(10, 595, 780, 14);
97     getContentPane().add(lblNodeDikunjungi_2007);
98
99     lblJumlahNode_2007 = new JLabel("Jumlah Node Dikunjungi: 0");
100    lblJumlahNode_2007.setBounds(10, 612, 780, 14);
101    getContentPane().add(lblJumlahNode_2007);
102
103    displayGraph_2007();
104 }
105
106 private void initializeGraph_2007() {
107     vertices_2007 = new HashMap<>();
108     adjacencyList_2007 = new HashMap<>();
109
110     addVertex_2007("Kantin DPR", 240, 60);
111     addVertex_2007("ATB", 100, 110);
112     addVertex_2007("DTI", 380, 110);
113
114     addVertex_2007("Gedung Audit", 240, 190);
115     addVertex_2007("FISIP", 100, 280);
116     addVertex_2007("Gedung H", 380, 280);
117
118     addVertex_2007("Kos A", 80, 390);
119     addVertex_2007("Kos B", 280, 390);
120     addVertex_2007("Aciak Mart", 320, 390);
121     addVertex_2007("Feb Mart", 440, 390);
122
123     addEdge_2007("Kantin DPR", "ATB");
124     addEdge_2007("Kantin DPR", "DTI");
125     addEdge_2007("Kantin DPR", "Gedung Audit");
126
127     addEdge_2007("ATB", "Gedung Audit");
128     addEdge_2007("DTI", "Gedung H");
129     addEdge_2007("DTI", "FISIP");
130
131     addEdge_2007("Gedung Audit", "Gedung H");
132     addEdge_2007("Gedung Audit", "Kos B");
133
134     addEdge_2007("Gedung Audit", "Kos B");
135     addEdge_2007("FISIP", "Kos A");
136     addEdge_2007("FISIP", "Kos B");
137
138     addEdge_2007("Kos A", "Kos B");
139     addEdge_2007("Kos B", "Aciak Mart");
140     addEdge_2007("Aciak Mart", "Feb Mart");
141     addEdge_2007("Feb Mart", "Gedung H");
142     addEdge_2007("Aciak Mart", "Gedung H");
143 }
144 private void addVertex_2007(String name_2007, int x_2007, int y_2007) {
145     vertices_2007.put(name_2007, new Vertex_2007(name_2007, x_2007, y_2007));
146     adjacencyList_2007.put(name_2007, new ArrayList<>());
147 }
148
149 private void addEdge_2007(String v1_2007, String v2_2007) {
150     adjacencyList_2007.get(v1_2007).add(v2_2007);
151     adjacencyList_2007.get(v2_2007).add(v1_2007);
152 }
153
154
155 public void BFS_2007() {
156     String start_2007 = (String) comboBoxAwal_2007.getSelectedItem();
157     String goal_2007 = (String) comboBoxAkhir_2007.getSelectedItem();
158
159     if (start_2007 == null || goal_2007 == null || start_2007.equals(goal_2007)) {
160         JOptionPane.showMessageDialog(this, "Pilih lokasi awal dan tujuan yang berbeda!");
161         return;
162     }
163
164     Queue<String> queue_2007 = new LinkedList<>();
165     Set<String> visited_2007 = new LinkedHashSet<>();
166     Map<String, String> parent_2007 = new HashMap<>();
167
168     queue_2007.add(start_2007);
169     visited_2007.add(start_2007);
170     boolean found_2007 = false;
171
172     while (!queue_2007.isEmpty()) {
173         String current_2007 = queue_2007.poll();
174

```

```

174         if (current_2007.equals(goal_2007)) {
175             found_2007 = true;
176             break;
177         }
178     }
179
180     List<String> neighbors_2007 = new ArrayList<>(adjacencyList_2007.get(current_2007));
181     Collections.sort(neighbors_2007);
182
183     for (String neighbor_2007 : neighbors_2007) {
184         if (!visited_2007.contains(neighbor_2007)) {
185             visited_2007.add(neighbor_2007);
186             parent_2007.put(neighbor_2007, current_2007);
187             queue_2007.add(neighbor_2007);
188         }
189     }
190 }
191
192 if (found_2007) {
193     List<String> path_2007 = new ArrayList<>();
194     String curr_2007 = goal_2007;
195     while (curr_2007 != null) {
196         path_2007.add(curr_2007);
197         curr_2007 = parent_2007.get(curr_2007);
198     }
199     Collections.reverse(path_2007);
200     visitedNodes_2007 = new ArrayList<>(visited_2007);
201     pathNodes_2007 = path_2007;
202     displayPath_2007("BFS", path_2007, visitedNodes_2007);
203 } else {
204     JOptionPane.showMessageDialog(this, "Jalur tidak ditemukan!");
205 }
206 }
207
208 public void DFS_2007() {
209     String start_2007 = (String) comboBoxAwal_2007.getSelectedItem();
210     String goal_2007 = (String) comboBoxAkhir_2007.getSelectedItem();
211
212     if (start_2007 == null || goal_2007 == null || start_2007.equals(goal_2007)) {
213         JOptionPane.showMessageDialog(this, "Pilih lokasi awal dan tujuan yang berbeda!");
214         return;
215     }
216
217     Stack<String> stack_2007 = new Stack<>();
218     Set<String> visited_2007 = new LinkedHashSet<>();
219     Map<String, String> parent_2007 = new HashMap<>();
220
221     stack_2007.push(start_2007);
222     visited_2007.add(start_2007);
223     boolean found_2007 = false;
224
225     while (!stack_2007.isEmpty()) {
226         String current_2007 = stack_2007.pop();
227
228         if (current_2007.equals(goal_2007)) {
229             found_2007 = true;
230             break;
231         }
232
233         List<String> neighbors_2007 = new ArrayList<>(adjacencyList_2007.get(current_2007));
234         Collections.sort(neighbors_2007);
235         Collections.reverse(neighbors_2007);
236
237         for (String neighbor_2007 : neighbors_2007) {
238             if (!visited_2007.contains(neighbor_2007)) {
239                 visited_2007.add(neighbor_2007);
240                 parent_2007.put(neighbor_2007, current_2007);
241                 stack_2007.push(neighbor_2007);
242             }
243         }
244     }
245
246     if (found_2007) {
247         List<String> path_2007 = new ArrayList<>();
248         String curr_2007 = goal_2007;
249         while (curr_2007 != null) {
250             path_2007.add(curr_2007);
251             curr_2007 = parent_2007.get(curr_2007);
252         }
253         Collections.reverse(path_2007);
254         visitedNodes_2007 = new ArrayList<>(visited_2007);
255         pathNodes_2007 = path_2007;
256         displayPath_2007("DFS", path_2007, visitedNodes_2007);
257     } else {
258         JOptionPane.showMessageDialog(this, "Jalur tidak ditemukan!");

```

```

258         JOptionPane.showMessageDialog(this, "Jalur tidak ditemukan!");
259     }
260 }
261
262 public void displayGraph_2007() {
263     panelGrafik_2007.repaint();
264 }
265
266 public void displayPath_2007(String method_2007, List<String> path_2007, List<String> visited_2007) {
267     StringBuilder hasil_2007 = new StringBuilder();
268     hasil_2007.append("<html><body style= width: 250px;'>");
269     hasil_2007.append("<b>Metode: </b>").append(method_2007).append("<br><br>");
270     hasil_2007.append("<b>Jalur Ditemukan:</b><br>");
271     hasil_2007.append(String.join(" &nbsp; ", path_2007)).append("<br><br>");
272     hasil_2007.append("<b>Urutan Node Dikunjungi:</b><br>");
273     hasil_2007.append(String.join(" ", visited_2007)).append("<br><br>");
274     hasil_2007.append("<b>Jumlah Node Dikunjungi:</b> ").append(visited_2007.size());
275     hasil_2007.append("</body></html>");
276
277     lblJalur_2007.setText("Jalur: " + String.join(" -> ", path_2007));
278     lblNodeDikunjungi_2007.setText("Node Dikunjungi: " + String.join(" ", visited_2007));
279     lblJumlahNode_2007.setText("Jumlah Node Dikunjungi: " + visited_2007.size());
280
281     displayGraph_2007();
282 }
283
284 public void resetGraph_2007() {
285     visitedNodes_2007 = new ArrayList<>();
286     pathNodes_2007 = new ArrayList<>();
287     lblJalur_2007.setText("Jalur: -");
288     lblNodeDikunjungi_2007.setText("Node Dikunjungi: -");
289     lblJumlahNode_2007.setText("Jumlah Node Dikunjungi: 0");
290     displayGraph_2007();
291 }
292
293
294 private static class Vertex_2007 {
295     String name_2007;
296     int x_2007, y_2007;
297     Color color_2007 = Color.WHITE;
298
299     Vertex_2007(String name_2007, int x_2007, int y_2007) {
300         this.name_2007 = name_2007;
301         this.x_2007 = x_2007;
302         this.y_2007 = y_2007;
303     }
304 }
305
306 private class GraphPanel_2007 extends JPanel {
307     @Override
308     protected void paintComponent(Graphics g) {
309         super.paintComponent(g);
310         Graphics2D g2d_2007 = (Graphics2D) g;
311         g2d_2007.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
312
313         g2d_2007.setColor(Color.GRAY);
314         g2d_2007.setStroke(new BasicStroke(2));
315         for (String v1_2007 : adjacencyList_2007.keySet()) {
316             Vertex_2007 vertex1_2007 = vertices_2007.get(v1_2007);
317             for (String v2_2007 : adjacencyList_2007.get(v1_2007)) {
318                 Vertex_2007 vertex2_2007 = vertices_2007.get(v2_2007);
319                 g2d_2007.drawLine(vertex1_2007.x_2007, vertex1_2007.y_2007, vertex2_2007.x_2007, vertex2_2007.y_2007);
320             }
321         }
322
323         for (Vertex_2007 v_2007 : vertices_2007.values()) {
324             if (pathNodes_2007 != null && pathNodes_2007.contains(v_2007.name_2007)) {
325                 v_2007.color_2007 = Color.GREEN;
326             } else if (visitedNodes_2007 != null && visitedNodes_2007.contains(v_2007.name_2007)) {
327                 v_2007.color_2007 = Color.YELLOW;
328             } else {
329                 v_2007.color_2007 = Color.WHITE;
330             }
331
332             g2d_2007.setColor(v_2007.color_2007);
333             g2d_2007.fillOval(v_2007.x_2007 - 25, v_2007.y_2007 - 25, 50, 50);
334             g2d_2007.setColor(Color.BLACK);
335             g2d_2007.setStroke(new BasicStroke(2));
336             g2d_2007.drawOval(v_2007.x_2007 - 25, v_2007.y_2007 - 25, 50, 50);
337
338             g2d_2007.setFont(new Font("SansSerif", Font.BOLD, 11));
339             FontMetrics fm_2007 = g2d_2007.getFontMetrics();
340             int textWidth_2007 = fm_2007.stringWidth(v_2007.name_2007);
341             g2d_2007.drawString(v_2007.name_2007, v_2007.x_2007 - (textWidth_2007 / 2), v_2007.y_2007 + 40);
342         }
343
344         g2d_2007.setFont(new Font("SansSerif", Font.BOLD, 11));
345         FontMetrics fm_2007 = g2d_2007.getFontMetrics();
346         int textWidth_2007 = fm_2007.stringWidth(v_2007.name_2007);
347         g2d_2007.drawString(v_2007.name_2007, v_2007.x_2007 - (textWidth_2007 / 2), v_2007.y_2007 + 40);
348     }
349 }
350 }

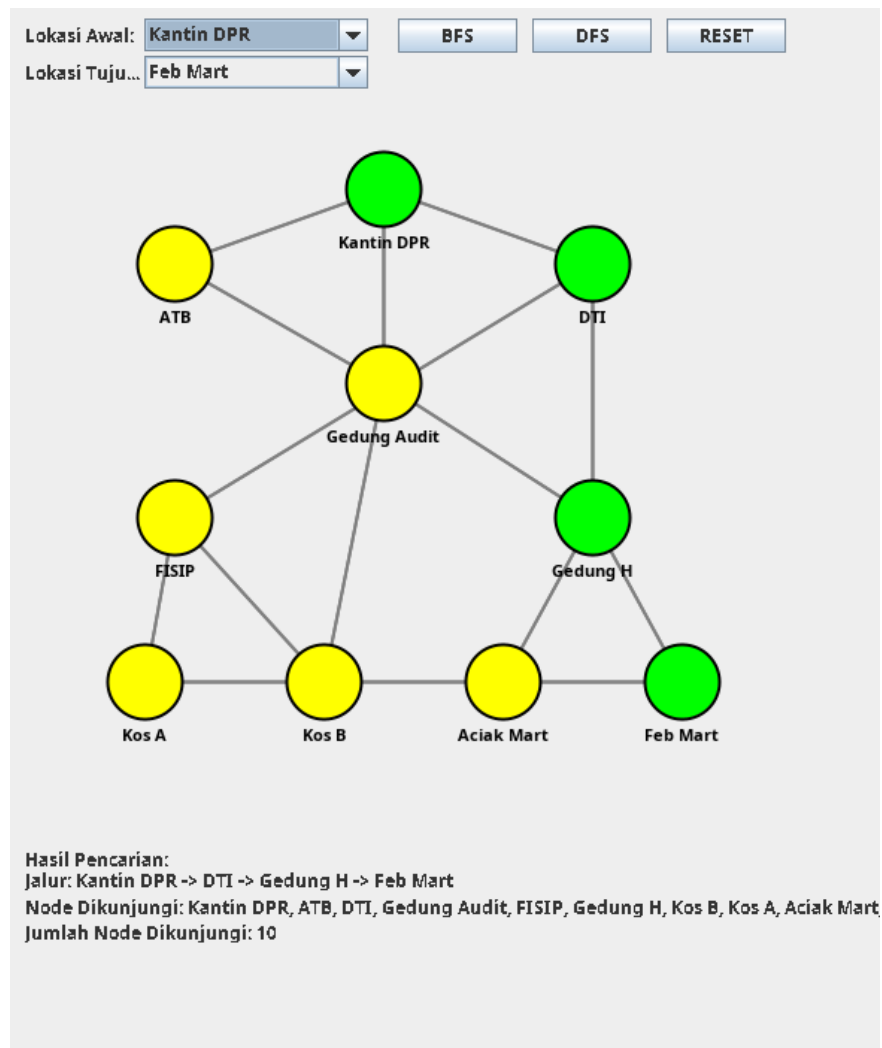
```

Penjelasan

1. initializeGraph_2007() membuat vertex lokasi dan edge penghubung.
2. addVertex_2007() menyimpan nama serta koordinat lokasi.

3. `addEdge_2007()` menambahkan relasi dua arah pada adjacency list.
4. `BFS_2007()` menggunakan queue dan parent map untuk memperoleh jalur.
5. `DFS_2007()` menggunakan stack untuk menelusuri graph.
6. `displayPath_2007()` menampilkan jalur dan node yang dikunjungi.
7. `GraphPanel_2007.paintComponent()` menggambar edge, node, nama lokasi, serta warna hasil pencarian.

Hasil Eksekusi Laporan Tugas



Analisis Hasil

Pada pengujian, lokasi awal Kantin DPR dan tujuan Feb Mart dipilih. BFS menelusuri graph menggunakan queue serta menyimpan parent setiap vertex untuk membentuk kembali jalur ketika tujuan ditemukan.

Node pada jalur ditandai hijau, sedangkan node lain yang sempat dikunjungi ditandai kuning. Label di bawah peta menampilkan jalur, urutan node dikunjungi, dan jumlah node sehingga hasil algoritma dapat diverifikasi.

Kesimpulan Laporan Tugas

petaHarian_2411532007 berhasil merepresentasikan peta sebagai graph tidak berarah dan menerapkan BFS serta DFS untuk pencarian jalur. Visualisasi memberikan informasi yang jelas mengenai rute dan proses penelusuran.

Saran

Aplikasi dapat dikembangkan dengan bobot jarak pada setiap edge, algoritma Dijkstra untuk rute terpendek berdasarkan jarak, tampilan peta yang lebih proporsional, serta pilihan untuk membandingkan hasil BFS dan DFS secara berdampingan.